

SPIRIT KNIGHT

Technical Documentation

Nguyễn Mạnh Cường

Vũ Việt Dũng

Trần Minh Tuấn Kiệt

Ngày 12 tháng 8 năm 2026

Mục lục

1	Giới thiệu	4
1.1	Tổng quan	4
1.2	Mục tiêu dự án	4
1.3	Sơ lược về cốt truyện và gameplay	4
1.4	Hướng dẫn chơi	4
2	Kiến trúc hệ thống	5
2.1	Tổng quan	5
2.2	Hệ thống mã nguồn	5
2.3	Class Main.java	6
2.3.1	Khởi tạo giao diện JavaFX	6
2.3.2	Khởi tạo hệ thống Input	7
2.3.3	Khởi tạo GameWorld và UIManager	7
2.3.4	Khởi tạo Game Loop	7
2.3.5	Khởi tạo cơ sở dữ liệu bất đồng bộ	7
3	Hệ thống GameWorld	8
3.1	Tổng quan	8
3.2	Kiến trúc tổng thể	8
3.3	Các thành phần chính	9
3.3.1	Các Manager	9
3.4	Quản lý trạng thái trò chơi	9
3.4.1	Chuyển đổi trạng thái	10
3.5	Chu trình cập nhật GameWorld	10
3.6	Cập nhật gameplay	11
3.7	Quản lý Player và Pet	12
3.7.1	Pet System	12
3.8	Hệ thống chiến đấu	12
3.8.1	Quản lý Bullet	12
3.8.2	Damage	13
3.9	Hệ thống kỹ năng và hiệu ứng chiến đấu	13
3.10	Quản lý Item	14
3.11	Hệ thống Room và Reward	14
3.12	Hệ thống vũ khí trong một lượt chơi	14
3.13	Khởi tạo một lượt chơi	15
3.14	Render thế giới	15
3.14.1	Y-Sorting	15
3.14.2	Render Pipeline	15

3.15	Camera	16
3.16	Hệ thống Save và Load	16
3.16.1	Database Worker	17
3.16.2	Cloud Save	17
3.16.3	Cloud Load	17
3.17	Auto Save	17
3.18	Quản lý Gold và Gem	17
3.19	Quản lý đồng thời	18
3.20	Quá trình Shutdown	18
3.21	Luồng hoạt động tổng quát	18
3.22	Kết luận	19
4	Hệ thống xử lý va chạm	20
4.1	Kiến trúc xử lý va chạm	20
4.2	Va chạm giữa thực thể và môi trường	21
4.2.1	Phương thức kiểm tra vị trí hợp lệ	21
4.3	Va chạm Player với tường và vật cản	22
4.3.1	Class xử lý	22
4.4	Va chạm Enemy với môi trường	23
4.4.1	Các phương thức liên quan	23
4.5	Va chạm Enemy với Enemy	24
4.5.1	Class và phương thức xử lý	24
4.6	Va chạm Player với Enemy	24
4.6.1	Class và phương thức xử lý	24
4.7	Xử lý sát thương của Player	25
4.8	Va chạm của Bullet	26
4.8.1	Phương thức điều phối	26
4.8.2	Bullet va chạm với tường	26
4.8.3	Bullet va chạm với Obstacle	27
4.8.4	Bullet của Player va chạm Enemy	27
4.8.5	Bullet của Enemy va chạm Player	28
4.9	Va chạm Player với Item	28
4.9.1	Class và phương thức	28
4.10	Tổng hợp các phương thức xử lý va chạm	30
4.11	Đánh giá thiết kế	30
4.12	Kết luận	30
5	Kĩ thuật Story cinematic	31
5.0.1	Điều phối cinematic theo chuỗi cảnh	31
5.0.2	Sử dụng JavaFX Animation	31
5.0.3	Hiệu ứng camera điện ảnh	32
5.0.4	Hiệu ứng hội thoại Visual Novel	32
5.0.5	Kết hợp hiệu ứng hình ảnh và âm thanh	33
5.0.6	Hiệu ứng Screen Shake và Flash	34
5.0.7	Tách dữ liệu cốt truyện khỏi mã nguồn	34
5.0.8	Cơ chế Skip và Continue	34
5.0.9	Tổng kết	35

6	Hệ thống cơ sở dữ liệu	36
6.1	Cơ chế Cache dữ liệu	37
7	Hệ thống giao diện người dùng	39
7.0.1	HUD trong gameplay	40
7.0.2	Giao diện Shop	40
7.0.3	Animation và Loading UI	40
7.0.4	Tổng kết	40
8	Hệ thống Buff	42
9	Hệ thống Weapon	44
9.0.1	Vũ khí tầm xa	44
9.0.2	Vũ khí cận chiến	45
9.0.3	Vũ khí tích năng	45
9.0.4	Quản lý loại vũ khí	45
9.0.5	Quản lý trang bị	46
10	Hệ thống Pet	47
10.0.1	Cơ chế di chuyển và theo Player	47
10.0.2	AI chiến đấu	48
10.0.3	Quản lý loại Pet	48
10.0.4	Animation	49
10.0.5	Pet khi chuyển phòng	49
10.0.6	Tổng kết	49
11	Hệ thống Item	50
11.0.1	Thu thập Item	50
11.0.2	Cơ chế hút Item	51
11.0.3	Hiệu ứng hiển thị	51
11.0.4	Tổng kết	51

Chương 1

Giới thiệu

1.1 Tổng quan

Spirit Knight là một game nhập vai phát triển bằng Java và JavaFX.

1.2 Mục tiêu dự án

Mục tiêu của dự án là củng cố các kiến thức lập trình nâng cao, quản lí logic , xây dựng tư duy hướng đối tượng , kĩ năng làm việc nhóm , sử dụng git

1.3 Sơ lược về cốt truyện và gameplay

Lấy cảm hứng từ tựa game Soul Knight - Chillyroom . Dự án được phát triển bởi nhóm sinh viên và học viên cao học ngành công nghệ thông tin .

Bối cảnh bắt đầu từ một không gian quen thuộc với các Dev đó là một buổi đêm muộn đêm bàn máy. Trong cái đêm định mệnh ấy bạn nhận được một thông điệp lạ và lời thỉnh cầu đi giải cứu trái đất đang bị các thể lực đen tối dòm ngó. Bạn là người được chọn và hành trình của bạn sẽ bắt đầu sau cánh cổng portal

1.4 Hướng dẫn chơi

Cần đăng kí đăng nhập trước khi chơi,dữ liệu người chơi sẽ được lưu lại trên cloud

Để di chuyển Player: sử dụng WASD hoặc các dấu mũi tên , bắn và thao tác bằng chuột trái, một số vũ khí cần nạp năng lượng bằng cách ấn giữ chuột trái,thả tay ra là bắn

Pause game: ESC hoặc tương tác trên màn hình

Tắt tiếng: M hoặc Pause -> Setting

Dùng buff: 1-2-3-4 hoặc ấn trực tiếp trên màn hình

Đổi vũ khí: Bên góc phải màn hình

Chế độ rảnh tay: bằng Touchpad để di chuyển 360độ và tự động ngắm bắn

Chương 2

Kiến trúc hệ thống

2.1 Tổng quan

Game được chia thành nhiều hệ thống như Player, Enemy, Weapon, Pet, Buff, Item, Database và UI

2.2 Hệ thống mã nguồn

Game được chia thành các package đảm bảo đưa các class liên quan về 1 package

- Package **animation**: gồm các class quản lý các hiệu ứng của game như: player/quái sinh/chết, render floating text, các hiệu ứng Particle pixel (các loại hạt), đổ bóng nhân vật
- Package **buff**: gồm danh sách các buff của game, tính năng của từng loại trong các class riêng, Package nhỏ **effect** phụ trách các hiệu ứng gây ra bởi các buff khi tấn công trực tiếp lên kẻ thù - các hiệu ứng của buff chủ yếu dùng render của javafx làm nền
- Package **cache**: phụ trách việc cache dữ liệu trong RAM, do game chạy bằng DB trực tuyến nên gây trễ khi chơi, việc cache sẽ phần nào giúp game chạy nhanh hơn và tăng trải nghiệm người chơi.
- Package **cinematic**: một package khá đặc biệt, nó góp phần tạo nên một phần intro story rất điện ảnh.
- Package **Database**: Linh hồn của dữ liệu game, lưu trữ hệ thống cơ sở dữ liệu của game sử dụng dịch vụ server của AVIEN dựa trên MySQL, tạo các truy vấn lên DB để lưu lại tên người chơi room họ đang chơi trước khi thoát, số điểm, số vàng họ kiếm được, các vật phẩm họ đã trang bị trong shop, các slot vũ khí họ sử dụng, tự động đồng bộ dữ liệu mỗi 15s
- Package **engine**: Đây là phần quan trọng nhất của game nơi chứa class **GameWorld.java** nơi gọi toàn bộ logic render, update player, enemy, weapon, bullet, vật cản, pet, quản lý va chạm giữa các thành phần gọi các hiệu ứng của toàn bộ vòng lặp game. Đồng thời chứa class **Camera.java** quản lý góc nhìn của game với trung tâm là player, class **InputHandler.java** phụ trách việc tiếp nhận dữ liệu đầu vào để điều khiển game - đặc biệt là có thể chơi cảm ứng với touchpad.

- Package **entity**: quản lí các đối tượng chính **Player** và **Enemy** kế thừa từ **Entity** của game. Quản lí các loại hero người chơi có thể lựa chọn, mua của Shop . Đồng thời cũng quản lí các loại quái, logic quái tự điều khiển để đánh Player.
- Package **item**: quản lí các vật phẩm nhặt được trong màn chơi, trong đó có gold, gem, và buff
- Package **map**: phụ trách việc load map từ file json, gán các chỉ số trong tiltmap với các assets, quản lí các **Room** và logic đóng mở cửa khi hoàn thành mission đánh quái ở từng phòng
- Package **pet**: Quản lí pet, các logic về pet- người bạn đồng hành trợ giúp Player đánh quái và các con pet để mua trong shop.
- Package **ui**: Quản lí toàn bộ giao diện game từ *intro, login, register* đến *main menu shop, leaderboard, setting, HUD*. Đồng thời cũng đồng bộ dữ liệu với Database đặc trưng riêng cho từng người dùng. Trái tim là class **UIManager.java** gọi toàn bộ logic UI từ các file **controler** và các file giao diện **fxml** kết hợp với style trong các file **CSS** cùng với DB quản lí các giao diện chuyên biệt với từng tài khoản khác nhau như chơi lần đầu, đợi load dữ liệu, ...
- Package **utils**: Quản lí các tiện ích cơ bản như các hằng số, load âm thanh, DatabaseExcuter và vector2D
- Package **wapon**: quản lý hệ thống vũ khí, các loại vũ khí được load lên shop và logic chiến đấu của từng loại.
- Class **Main.java** là điểm khởi đầu (entry point) của ứng dụng Spirit Knight. Lớp này kế thừa **Application** của JavaFX và chịu trách nhiệm khởi tạo các thành phần chính cần thiết trước khi trò chơi bắt đầu hoạt động.
Các trách nhiệm chính của lớp **Main** bao gồm:
 - Khởi tạo cửa sổ JavaFX và vùng render của game.
 - Khởi tạo hệ thống xử lý input.
 - Khởi tạo **GameWorld**.
 - Khởi tạo và liên kết **UIManager** với game.
 - Khởi tạo và điều khiển **GameLoop**.
 - Khởi tạo kết nối cơ sở dữ liệu trên một luồng riêng.
 - Quản lý quá trình lưu dữ liệu và giải phóng tài nguyên khi người chơi thoát game.

2.3 Class Main.java

2.3.1 Khởi tạo giao diện JavaFX

Khi ứng dụng được khởi chạy, JavaFX gọi phương thức **start(Stage stage)**. Đây là nơi lớp **Main** khởi tạo cửa sổ chính của trò chơi. Một đối tượng **Canvas** được tạo với kích thước mặc định được định nghĩa trong lớp **Constants**. Canvas đóng vai trò là bề mặt chính để render các thành phần của thế giới game.

```
Canvas canvas = new Canvas(  
    Constants.WINDOW_WIDTH,  
    Constants.WINDOW_HEIGHT  
);
```

```
GraphicsContext gc = canvas.getGraphicsContext2D();
```

Đối tượng `GraphicsContext` được lấy từ `Canvas` và được sử dụng bởi hệ thống render để vẽ các thành phần của trò chơi.

2.3.2 Khởi tạo hệ thống Input

Lớp `InputHandler` chịu trách nhiệm tiếp nhận các sự kiện đầu vào từ người chơi như bàn phím và các thao tác điều khiển được hỗ trợ bởi game. `InputHandler` được liên kết với `Scene` để nhận các sự kiện đầu vào và sau đó được truyền vào `GameWorld`.

2.3.3 Khởi tạo `GameWorld` và `UIManager`

Sau khi hệ thống input được thiết lập, `Main` khởi tạo đối tượng `GameWorld`.

```
GameWorld world = new GameWorld(inputHandler);
```

`GameWorld` đóng vai trò quản lý trạng thái và các thành phần chính của thế giới game. Tiếp theo, hệ thống giao diện được khởi tạo thông qua `UIManager`.

Việc liên kết `UIManager` với `GameWorld` cho phép giao diện truy cập các trạng thái cần thiết của game để cập nhật HUD và các thành phần giao diện khác.

2.3.4 Khởi tạo Game Loop

Lớp `Main` khởi tạo `GameLoop` để thực hiện vòng lặp chính của trò chơi. Mỗi vòng lặp nhận giá trị `delta`, biểu diễn khoảng thời gian giữa hai frame liên tiếp.

Trong mỗi frame, hệ thống thực hiện ba nhiệm vụ chính:

1. Xử lý input toàn cục.
2. Cập nhật trạng thái thế giới game.
3. Render thế giới và cập nhật HUD.

2.3.5 Khởi tạo cơ sở dữ liệu bất đồng bộ

Việc kiểm tra và khởi tạo cơ sở dữ liệu được thực hiện trên một luồng riêng thay vì trực tiếp trên `JavaFX Application Thread`. Thiết kế này giúp quá trình kết nối cơ sở dữ liệu không chặn `JavaFX Application Thread`, từ đó hạn chế tình trạng giao diện bị đứng trong quá trình khởi động game. Luồng này được thiết lập là `daemon thread` để không ngăn JVM kết thúc khi ứng dụng được đóng.

Chương 3

Hệ thống GameWorld

3.1 Tổng quan

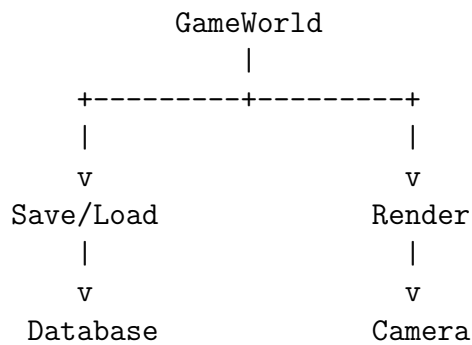
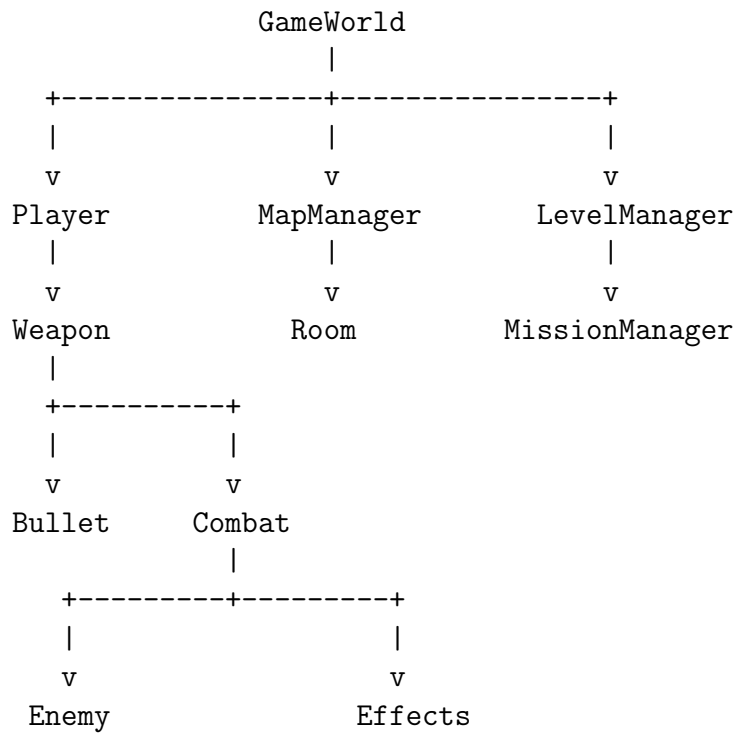
Lớp `GameWorld` thuộc package `com.soulknight.engine` và đóng vai trò là bộ điều phối trung tâm của thế giới trò chơi `Spirit Knight`. Khác với lớp `Main` chịu trách nhiệm khởi tạo ứng dụng, `GameWorld` quản lý trạng thái của một phiên chơi và điều phối hoạt động giữa các hệ thống gameplay. Các thành phần được quản lý bởi `GameWorld` bao gồm:

- Player và Pet.
- Enemy và quá trình spawn Enemy.
- Map, Room và Obstacle.
- Bullet và hệ thống va chạm.
- Weapon và loadout của lượt chơi.
- Item và quá trình thu thập vật phẩm.
- Buff và các hiệu ứng chiến đấu.
- Particle và các hiệu ứng hình ảnh.
- Mission và Reward.
- Camera và quá trình render thế giới.
- Game State.
- Save/Load dữ liệu người chơi.

Có thể xem `GameWorld` như lớp trung gian kết nối các subsystem gameplay với vòng lặp chính của trò chơi.

3.2 Kiến trúc tổng thể

Về mặt kiến trúc, `GameWorld` không trực tiếp cài đặt toàn bộ hành vi của từng đối tượng. Thay vào đó, lớp giữ tham chiếu đến các manager và entity, sau đó điều phối chúng trong quá trình update và render. Mô hình tổng quát có thể biểu diễn như sau:



Thiết kế này cho phép các hệ thống chuyên biệt như **MapManager**, **LevelManager**, **MissionManager** hay **CombatEffectManager** tập trung vào trách nhiệm riêng, trong khi **GameWorld** quyết định thời điểm chúng được cập nhật.

3.3 Các thành phần chính

3.3.1 Các Manager

GameWorld sở hữu nhiều manager phục vụ cho các hệ thống khác nhau của game.

3.4 Quản lý trạng thái trò chơi

Một trong những trách nhiệm quan trọng nhất của **GameWorld** là quản lý trạng thái hiện tại của trò chơi thông qua **GameState**. Các trạng thái được xử lý trong **GameWorld** bao gồm:

- INTRO

Thành phần	Trách nhiệm
LevelManager	Quản lý level và tiến trình của một lượt chơi.
MapManager	Quản lý bản đồ, tile, room, portal và dữ liệu không gian.
MissionManager	Quản lý nhiệm vụ của level hiện tại.
EnemyFactory	Khởi tạo các Enemy phù hợp với level.
ParticleManager	Quản lý các particle effect trong game.
EnemyBurnManager	Quản lý trạng thái Burn của Enemy.
CombatEffectManager	Quản lý các hiệu ứng chiến đấu đặc biệt.
ItemMagnetSystem	Điều khiển cơ chế hút Item về phía Player.
FloatingTextManager	Hiển thị damage, gold và các thông báo nổi trong game.

Bảng 3.1: Một số subsystem được GameWorld quản lý

- MAIN_MENU
- PLAYING
- PAUSED
- LEVEL_CLEAR
- REWARD_PICK
- GAME_OVER
- GAME_VICTORY

3.4.1 Chuyển đổi trạng thái

Việc chuyển trạng thái được tập trung trong phương thức `changeState()`. Khi trạng thái thay đổi, trạng thái input cũ được xóa nhằm tránh việc một thao tác bàn phím hoặc chuột từ state trước tiếp tục ảnh hưởng tới state mới. Sau đó `GameStateListener` được thông báo để các hệ thống khác, đặc biệt là UI, có thể phản ứng với trạng thái mới.

3.5 Chu trình cập nhật GameWorld

GameWorld trong mỗi frame. Nếu trò chơi đang ở trạng thái PAUSED, phương thức kết thúc ngay lập tức và toàn bộ gameplay ngừng cập nhật. Đối với các trạng thái còn lại, GameWorld sử dụng `switch` để chuyển việc xử lý đến logic tương ứng.

```

GameLoop
|
v
GameWorld.update()
|
+-- PAUSED -----> Stop Update
|

```

```

+-- INTRO -----> Input
|
+-- MAIN_MENU -----> Input
|
+-- PLAYING -----> updatePlaying()
|
+-- LEVEL_CLEAR ----> updateLevelClear()
|
+-- REWARD_PICK ----> updateRewardPick()
|
+-- GAME_OVER -----> Restart
|
+-- GAME_VICTORY ----> Restart

```

3.6 Cập nhật gameplay

Khi state hiện tại là **PLAYING**, GameWorld gọi phương thức `updatePlaying()`. Đây là một trong những phương thức điều phối quan trọng nhất của hệ thống gameplay. Quá trình update được thực hiện theo thứ tự tổng quát:

1. Cập nhật hiệu ứng spawn và death.
2. Cập nhật Player.
3. Cập nhật Pet.
4. Xác định Room hiện tại.
5. Cập nhật trạng thái Room.
6. Cập nhật Enemy.
7. Xử lý va chạm Player–Enemy.
8. Cập nhật Bullet.
9. Xử lý Obstacle bị phá hủy.
10. Cập nhật các Combat Effect.
11. Cập nhật trạng thái Burn.
12. Cập nhật Item Magnet.
13. Kiểm tra Player thu thập Item.
14. Kiểm tra Reward.
15. Thực hiện Auto Save.
16. Cập nhật Camera.

17. Kiểm tra trạng thái sống/chết của Player.

18. Cập nhật Particle và Floating Text.

Thứ tự update có ý nghĩa quan trọng vì nhiều subsystem phụ thuộc vào kết quả của subsystem trước đó.

3.7 Quản lý Player và Pet

GameWorld giữ tham chiếu đến Player hiện tại thông qua lời gọi đến class Player. Trong trạng thái PLAYING, Player được cập nhật trước nhiều hệ thống chiến đấu khác. Việc truyền `this` vào `player.update()` cho phép Player tương tác với dữ liệu của thế giới như Enemy, Bullet, Map và các subsystem gameplay.

3.7.1 Pet System

Loại Pet được lấy từ `PetSelectionManager` và instance thực tế được tạo thông qua `PetFactory`. Nhờ đó, hệ thống lựa chọn Pet và đối tượng Pet tồn tại trong game được tách biệt với nhau.

3.8 Hệ thống chiến đấu

GameWorld đóng vai trò điều phối nhiều thành phần của Combat System, bao gồm:

- Bullet.
- Slash Effect.
- Explosion Effect.
- Ion Explosion.
- Sound Wave.
- Burn Effect.
- Damage.
- Combat Buff.

3.8.1 Quản lý Bullet

Các projectile đang tồn tại được lưu trong:

```
private final List<Bullet> bullets = new ArrayList<>();
```

Trong mỗi frame, `updateBullets()` chịu trách nhiệm cập nhật vị trí và kiểm tra va chạm. Một Bullet có thể tương tác với:

- Wall.
- Obstacle.

- Enemy.
- Player.

Tùy loại vũ khí, Bullet còn có thể có các hành vi đặc biệt như phản xạ, xuyên mục tiêu, tạo explosion hoặc tạo sound wave.

3.8.2 Damage

Khi Bullet của Player va chạm Enemy, GameWorld tính lượng damage thực tế dựa trên HP trước và sau khi nhận sát thương. Mô hình xử lý có dạng:

```

Bullet
|
v
Collision
|
v
Enemy.takeDamage()
|
+----> Floating Damage Text
|
+----> Particle Effect
|
+----> Buff Notification

```

Việc sử dụng lượng damage thực tế giúp hệ thống Buff nhận đúng lượng damage đã gây ra thay vì chỉ sử dụng damage lý thuyết của projectile.

3.9 Hệ thống kỹ năng và hiệu ứng chiến đấu

GameWorld còn điều phối các kỹ năng đặc biệt của Player-được kích hoạt khi sử dụng các Buffs Ví dụ, hệ thống hiện tại bao gồm:

- Dragon Breath.
- Holy Nova.
- Chain Lightning.
- Death Explosion.
- Ion Explosion.

Đối với Dragon Breath, GameWorld xác định Enemy nằm trong vùng hình nón trước mặt Player. Damage được xử lý trực tiếp bởi logic gameplay, trong khi animation chỉ đóng vai trò biểu diễn trực quan. Cách tách gameplay logic và visual effect này giúp kết quả combat không phụ thuộc vào thời gian chạy animation.

3.10 Quản lý Item

GameWorld sử dụng `ItemMagnetSystem` để cập nhật chuyển động hút Item về phía Player. Sau đó `updateItemCollection()` kiểm tra va chạm giữa Player và Item. Các loại Item hiện được xử lý bao gồm:

- Gold.
- Gem.
- Buff.
- Energy Crystal.

Sau khi Item được thu thập, GameWorld cập nhật dữ liệu tương ứng, tạo visual feedback và loại Item khỏi danh sách active.

3.11 Hệ thống Room và Reward

GameWorld theo dõi Room mà Player đang đứng. Trạng thái Room ở frame trước cũng được lưu lại để GameWorld có thể phát hiện sự thay đổi trạng thái của phòng. Sau khi một phòng chiến đấu được hoàn thành, hệ thống có thể mở `RewardPicker`. Reward hiện tại cho phép Player lựa chọn giữa:

- Gold.
- Weapon mới.

Mỗi phòng chỉ được nhận reward một lần trong cùng một lượt chơi.

3.12 Hệ thống vũ khí trong một lượt chơi

GameWorld quản lý hai slot vũ khí riêng của lượt chơi. Khi bắt đầu một run, loadout được snapshot từ `WeaponSelectionManager`. Nếu slot không có dữ liệu hợp lệ, hệ thống sử dụng vũ khí mặc định:

- Slot 1: BLASTER.
- Slot 2: OLD_SWORD.

Thiết kế này tạo ra sự phân biệt giữa:

```
Shop Loadout
|
| Start Run
v
Run Loadout
|
+---- Slot 1
|
+---- Slot 2
```

Do đó, việc nhận Weapon Reward trong dungeon có thể thay đổi vũ khí của run hiện tại mà không nhất thiết thay đổi lựa chọn vũ khí được lưu trong Shop.

3.13 Khởi tạo một lượt chơi

Khi bắt đầu một lượt chơi mới, GameWorld thực hiện quá trình khởi tạo thông qua `startNewRun()` và `loadCurrentLevel()`. Quá trình load level bao gồm:

1. Khởi tạo MapManager.
2. Load Obstacle từ Tile Map.
3. Xác định spawn point.
4. Khởi tạo hoặc di chuyển Player.
5. Khôi phục dữ liệu Save nếu có.
6. Khởi tạo Pet.
7. Tạo Spawn Effect.
8. Trang bị Weapon.
9. Xóa Entity và Effect của level trước.
10. Tạo Mission.
11. Khởi tạo Item đặc thù của level.
12. Spawn Boss nếu level hiện tại là Boss Level.

3.14 Render thế giới

Phần render gameplay chính được thực hiện bởi `renderWorld()`.

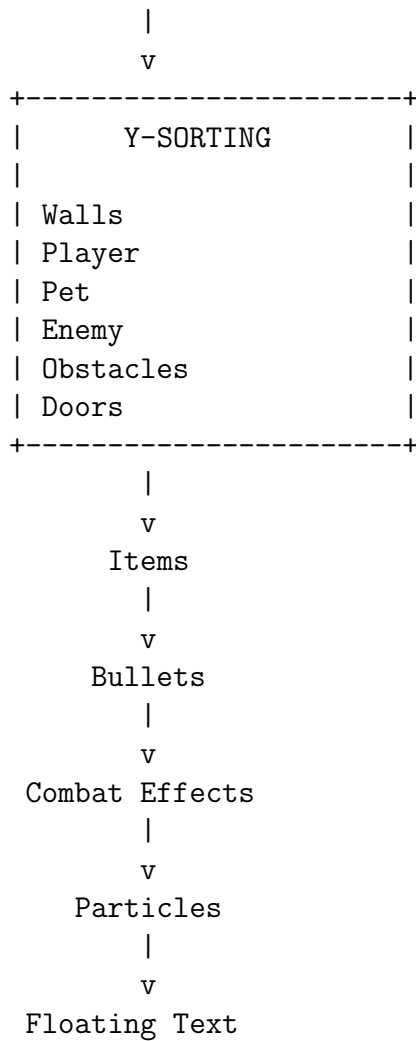
3.14.1 Y-Sorting

Spirit Knight sử dụng góc nhìn giả 2.5D. Vì vậy, thứ tự render không thể chỉ phụ thuộc vào loại Entity. GameWorld tạo một danh sách các đối tượng render và gán cho mỗi đối tượng một giá trị độ sâu dựa trên tọa độ Y. Những đối tượng có vị trí thấp hơn trên màn hình sẽ được render sau, tạo cảm giác chúng nằm phía trước đối tượng có tọa độ Y nhỏ hơn.

3.14.2 Render Pipeline

Pipeline render tổng quát có thể biểu diễn:

```
Dynamic Background
    |
    v
  Floor
    |
    v
Obstacle Shadows
```

3.15 Camera

GameWorld sở hữu một đối tượng **Camera** dùng để chuyển đổi tọa độ thế giới sang tọa độ màn hình. Trong quá trình gameplay, camera liên tục follow Player: Ngoài ra, GameWorld kiểm tra object có nằm trong vùng camera hay không trước khi render. Điều này giúp tránh thực hiện các draw call không cần thiết đối với object nằm ngoài viewport.

3.16 Hệ thống Save và Load

GameWorld tích hợp trực tiếp với hệ thống persistence thông qua:

- PlayerSave.
- PlayerSaveDAO.
- PlayerSaveMapper.
- ShopDAO.
- UserSession.

3.16.1 Database Worker

Các thao tác database được chuyển sang một luồng `ExecutorService` riêng.

```
private final ExecutorService databaseExecutor =  
    Executors.newSingleThreadExecutor(runnable -> {  
        Thread thread = new Thread(  
            runnable ,  
            "soul-knight-database-worker "  
        );  
  
        thread.setDaemon(true);  
        return thread;  
    });
```

Mục tiêu của thiết kế này là tránh thực hiện các thao tác I/O database trực tiếp trên game thread, từ đó giảm nguy cơ gây khựng hoặc đứng game.

3.16.2 Cloud Save

Phương thức `saveGameAsync()` chuyển trạng thái hiện tại của `GameWorld` thành `PlayerSave` thông qua `PlayerSaveMapper`. Sau đó dữ liệu được lưu bằng `PlayerSaveDAO` trên database worker.

3.16.3 Cloud Load

Quá trình load được thực hiện bất đồng bộ thông qua `loadGameAsync()`. Dữ liệu lấy từ database được lưu tạm trong:

```
private PlayerSave pendingPlayerSave;
```

Sau khi Player và level đã được khởi tạo, dữ liệu này mới được áp dụng trở lại `GameWorld`. Cách thực hiện này tránh việc restore dữ liệu vào Player trước khi Player thực sự tồn tại.

3.17 Auto Save

`GameWorld` định nghĩa khoảng thời gian Auto Save:

```
private static final double AUTO_SAVE_INTERVAL = 30.0;
```

Trong quá trình gameplay, bộ đếm thời gian được cập nhật theo `deltaSeconds`. Khi đạt đến khoảng thời gian quy định, `GameWorld` yêu cầu lưu trạng thái hiện tại. Việc save thực tế vẫn được chuyển sang database worker để không làm gián đoạn game loop.

3.18 Quản lý Gold và Gem

`GameWorld` phân biệt tài nguyên thu được trong run với tài nguyên đã được đồng bộ vào tài khoản. Các biến: lưu lượng tài nguyên đang chờ đồng bộ. Khi Player nhận Gold hoặc Gem, giá trị tương ứng được cộng vào pending bank. Sau đó `syncBankRewardsAsync()` chuyển lượng tài nguyên này sang database. Nếu quá trình database thất bại, lượng Gold và Gem chưa đồng bộ được trả lại vào pending bank để có thể thử lại ở lần sau. Cơ chế này giúp giảm nguy cơ mất reward khi xảy ra lỗi kết nối database.

3.19 Quản lý đồng thời

Do GameWorld vừa hoạt động trên game thread vừa giao tiếp với database worker, một số trạng thái cần được bảo vệ khỏi việc truy cập đồng thời. Các đối tượng `AtomicBoolean` được sử dụng để ngăn nhiều thao tác Save, Load hoặc Bank Sync chạy đồng thời. Ngoài ra, dữ liệu reward đang chờ đồng bộ được bảo vệ bởi một lock riêng. Thiết kế này đặc biệt quan trọng vì database worker hoạt động trên thread khác với gameplay.

3.20 Quá trình Shutdown

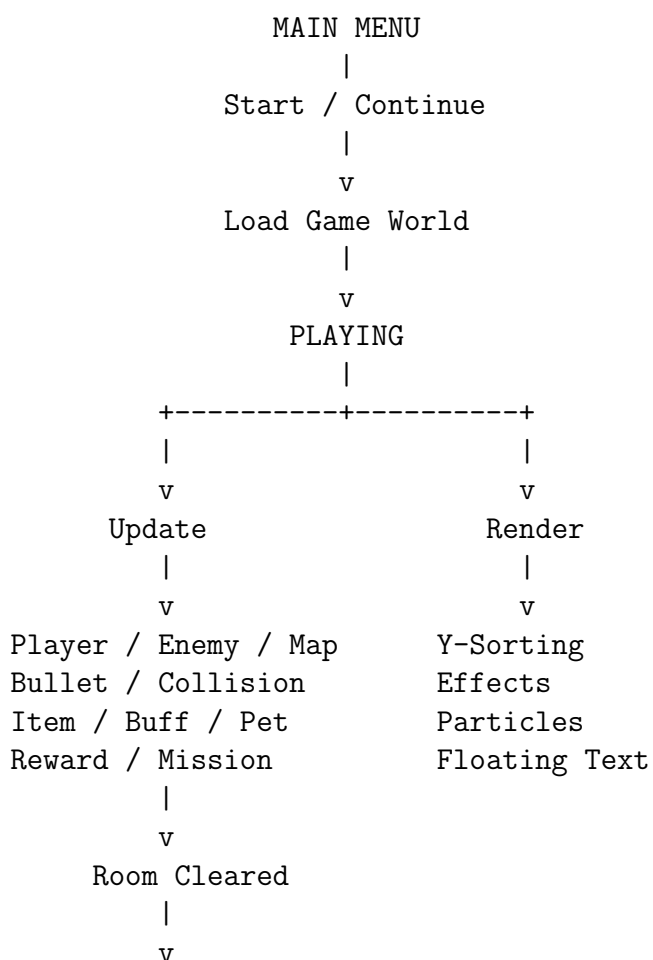
Khi ứng dụng chuẩn bị đóng, phương thức `shutdown()` được sử dụng để kết thúc các tài nguyên thuộc GameWorld. Quá trình này bao gồm:

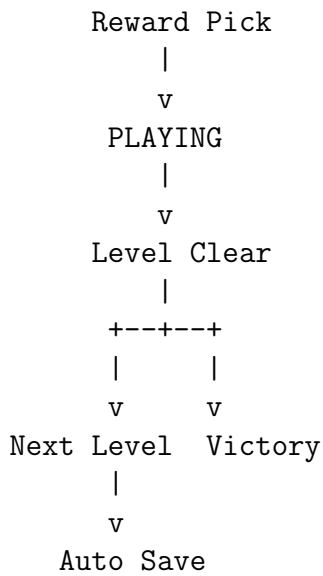
1. Đồng bộ Gold và Gem còn tồn đọng.
2. Yêu cầu database executor dừng hoạt động.

Điều này phối hợp với shutdown handler của lớp `Main` để hạn chế mất dữ liệu khi người chơi đóng game.

3.21 Luồng hoạt động tổng quát

Có thể tóm tắt vòng đời gameplay được điều phối bởi GameWorld như sau:





3.22 Kết luận

GameWorld là một trong những lớp trung tâm của kiến trúc Spirit Knight. Lớp kết nối Game Loop với hầu hết các subsystem gameplay và chịu trách nhiệm duy trì trạng thái của một lượt chơi. Thông qua GameWorld, quá trình cập nhật Player, Enemy, Bullet, Room, Item, Pet, Mission, Reward, Camera, Render và Save/Load được thực hiện theo một trình tự thống nhất trong mỗi frame. Vì vậy, có thể xem **GameWorld** là lớp điều phối gameplay chính của Spirit Knight.

Chương 4

Hệ thống xử lý va chạm

Hệ thống xử lý va chạm là một thành phần quan trọng trong quá trình mô phỏng thế giới game. Trong *Spirit Knight*, phần lớn quá trình điều phối va chạm được thực hiện tại lớp `GameWorld`, trong khi các lớp thực thể như `Player`, `Enemy`, `Bullet` và `Obstacle` cung cấp các phương thức hỗ trợ tương ứng.

Trong mỗi frame của game, phương thức:

```
GameWorld.updatePlaying(...)
```

thực hiện cập nhật các thực thể và lần lượt kích hoạt các hệ thống va chạm quan trọng. Các nhóm va chạm chính bao gồm:

- Player – tường và vật cản;
- Enemy – tường và vật cản;
- Enemy – Enemy;
- Player – Enemy;
- Bullet – tường;
- Bullet – Obstacle;
- Bullet – Enemy;
- Bullet – Player;
- Player – Item.

4.1 Kiến trúc xử lý va chạm

Có thể mô tả luồng xử lý tổng quát như sau:

```
GameWorld.updatePlaying()  
    |  
    +---> Player.update()  
    |      |  
    |      +---> Player.move()
```

```

|
|
|      +---> GameWorld.canMoveTo()
|
+---> Enemy.update()
|
|      +---> Enemy.move()
|      |
|      |      +---> GameWorld.canMoveTo()
|      |
|      +---> Enemy.separateFromOtherEnemies()
|
+---> GameWorld.resolvePlayerEnemyCollisions()
|
+---> GameWorld.updateBullets()
|
|      +---> Bullet - Wall
|      +---> Bullet - Obstacle
|      +---> Bullet - Enemy
|      +---> Bullet - Player
|
+---> GameWorld.updateItemCollection()

```

Như vậy, `GameWorld` đóng vai trò là lớp điều phối trung tâm, còn từng Entity chịu trách nhiệm cung cấp thông tin vị trí, bán kính, màu và trạng thái cần thiết cho việc kiểm tra va chạm.

4.2 Va chạm giữa thực thể và môi trường

4.2.1 Phương thức kiểm tra vị trí hợp lệ

Một phương thức quan trọng của hệ thống là:

```
GameWorld.canMoveTo(Vector2D position, double radius)
```

Phương thức này được sử dụng để xác định một thực thể có thể di chuyển đến vị trí mới hay không.

Quá trình kiểm tra gồm hai bước chính:

1. Kiểm tra vị trí mới có nằm trong khu vực có thể di chuyển của bản đồ hay không.
2. Kiểm tra hình tròn va chạm của thực thể có giao với `Obstacle` hay không.

Mã xử lý có dạng:

```

if (!mapManager.isWalkable(
    position.getX(),
    position.getY(),

```

```

        radius)) {
    return false;
}

for (Obstacle obstacle : obstacles) {

    if (obstacle == null || obstacle.isDestroyed()) {
        continue;
    }

    if (obstacle.intersectsCircle(position, radius)) {
        return false;
    }
}

return true;

```

Có thể biểu diễn điều kiện va chạm với môi trường dưới dạng:

$$Move(P, r) = Walkable(P, r) \wedge \neg CollisionObstacle(P, r)$$

trong đó:

- P là vị trí dự kiến của thực thể;
- r là bán kính va chạm;
- $Walkable(P, r)$ kiểm tra tường của bản đồ;
- $CollisionObstacle(P, r)$ kiểm tra các vật cản trong phòng.

4.3 Va chạm Player với tường và vật cản

4.3.1 Class xử lý

- Class thực thể: Player
- Class điều phối: GameWorld
- Phương thức di chuyển: Player.move()
- Phương thức kiểm tra: GameWorld.canMoveTo()

Trong quá trình cập nhật Player, vector di chuyển được tính từ input:

```

movement.scale(
    getSpeed() * weaponSpeedMultiplier * deltaSeconds
);

this.move(

```

```

    world,
    movement.getX(),
    movement.getY()
);

```

Trước khi cập nhật vị trí thật sự, vị trí mới phải vượt qua `GameWorld.canMoveTo()`. Cơ chế này giúp ngăn Player:

- đi xuyên tường;
- đi ra ngoài vùng hợp lệ;
- xuyên qua các Obstacle;
- đứng vào vùng không thể di chuyển.

4.4 Va chạm Enemy với môi trường

4.4.1 Các phương thức liên quan

- `Enemy.move()`
- `Enemy.smartMoveTo()`
- `GameWorld.canMoveTo()`
- `Obstacle.intersectsCircle()`

Enemy không trực tiếp thay đổi vị trí mà không kiểm tra va chạm. Trong phương thức:

```
Enemy.move(GameWorld world, double dx, double dy)
```

Enemy lần lượt thử:

1. di chuyển đồng thời theo trục X và Y ;
2. nếu bị chặn, thử riêng trục X ;
3. nếu vẫn bị chặn, thử riêng trục Y .

Pseudo-code:

```

newPosition = currentPosition + movement

if (world.canMoveTo(newPosition))
    move XY
else if (world.canMoveTo(positionXOnly))
    move X
else if (world.canMoveTo(positionYOnly))
    move Y

```

Cách xử lý này giúp Enemy có khả năng “trượt” dọc theo tường thay vì dừng hoàn toàn khi gặp vật cản.

4.5 Va chạm Enemy với Enemy

4.5.1 Class và phương thức xử lý

- **Class:** Enemy
- **Phương thức:** `separateFromOtherEnemies()`

Phương thức có dạng:

```
Enemy.separateFromOtherEnemies(  
    GameWorld world,  
    List<Enemy> allEnemies,  
    double deltaSeconds  
)
```

Khoảng cách giữa hai Enemy được tính:

$$d = \|P_1 - P_2\|$$

Khoảng cách tối thiểu cho phép là:

$$d_{min} = r_1 + r_2$$

Nếu:

$$d < d_{min}$$

thì hai Enemy đang chồng lên nhau.

Độ chồng lấn được xác định:

$$overlap = d_{min} - d$$

Sau đó một vector đẩy được tạo theo hướng từ Enemy còn lại đến Enemy hiện tại:

$$\vec{D} = \frac{P_1 - P_2}{\|P_1 - P_2\|}$$

Enemy được dịch chuyển theo hướng $\vec{D}$ để tách hai thực thể.

Cơ chế này đặc biệt quan trọng khi nhiều Enemy cùng truy đuổi Player, giúp tránh hiện tượng nhiều quái vật chồng sprite lên nhau.

4.6 Va chạm Player với Enemy

4.6.1 Class và phương thức xử lý

- **Class điều phối:** GameWorld
- **Phương thức:** `resolvePlayerEnemyCollisions()`
- **Class nhận sát thương:** Player

- **Phương thức sát thương:** `Player.takeDamage()`

Sau khi cập nhật Enemy, GameWorld gọi:

```
resolvePlayerEnemyCollisions(deltaSeconds);
```

để xử lý việc Player và Enemy tiếp xúc vật lý.

Ngoài cơ chế phân tách vị trí, Enemy cận chiến còn sử dụng khoảng cách giữa hai thực thể để xác định khả năng gây sát thương.

Khoảng cách an toàn được xác định gần tương đương:

$$d_{safe} = r_{player} + r_{enemy} + \varepsilon$$

với ε là khoảng đệm nhỏ.

Khi Enemy nằm trong phạm vi tấn công và thời gian hồi chiêu bằng không, Enemy gọi:

```
world.getPlayer().takeDamage(contactDamage);
```

Sau đó bộ đếm hồi chiêu được thiết lập để tránh gây sát thương liên tục trong từng frame.

4.7 Xử lý sát thương của Player

Player sử dụng:

```
Player.takeDamage(int amount)
```

để xử lý sát thương sau va chạm.

Phương thức này không chỉ giảm HP mà còn kiểm tra:

- trạng thái bất tử tạm thời;
- hệ số giảm sát thương từ Buff;
- Shield;
- lượng sát thương thực tế vào HP;
- Buff đặc biệt như Holy Nova.

Sau khi nhận sát thương thành công, Player kích hoạt khoảng thời gian bất tử ngắn:

```
invulnerabilityTimer = MAX_INVULNERABILITY_TIME;
```

Trong chương trình:

$$T_{invulnerability} = 0.3 \text{ s}$$

Cơ chế này ngăn Player nhận hàng chục lần sát thương khi hai hitbox tiếp xúc trong nhiều frame liên tiếp.

4.8 Va chạm của Bullet

4.8.1 Phương thức điều phối

Phần lớn va chạm liên quan đến đạn được xử lý tại:

```
GameWorld.updateBullets(double deltaSeconds)
```

Đây là một trong những phương thức xử lý va chạm quan trọng nhất trong `GameWorld`. Một viên đạn có thể tương tác với:

- tường;
- Obstacle;
- Enemy;
- Player.

4.8.2 Bullet va chạm với tường

Phương thức kiểm tra:

```
MapManager.isBulletCollidingWithWall(  
    bulletX,  
    bulletY,  
    bullet.getRadius()  
)
```

Nếu đạn thường chạm tường, đạn được vô hiệu hóa:

```
bullet.deactivate();
```

Đối với một số loại đạn đặc biệt, hệ thống có thể thực hiện hành vi khác. Ví dụ đạn có khả năng phản xạ sử dụng:

```
bullet.canReflect()  
bullet.reflectOnce(...)
```

Hệ thống xác định trục va chạm rồi thay đổi hướng chuyển động của viên đạn.

4.8.3 Bullet va chạm với Obstacle

Va chạm giữa đạn và vật cản sử dụng:

```
obstacle.intersectsCircle(  
    bullet.getPosition(),  
    bullet.getRadius()  
)
```

Nếu xảy ra va chạm, tùy thuộc thuộc tính của Bullet, hệ thống có thể:

- gây sát thương lên Obstacle;
- xuyên qua Obstacle;
- phát nổ;
- sinh hiệu ứng Sound Wave;
- vô hiệu hóa viên đạn.

Sát thương lên vật cản được thực hiện thông qua:

```
GameWorld.damageObstacle(...)
```

4.8.4 Bullet của Player va chạm Enemy

Khi chủ sở hữu của Bullet là Player:

```
if (bullet.getOwner() instanceof Player)
```

hệ thống duyệt danh sách Enemy và kiểm tra:

```
bullet.intersects(enemy)
```

Nếu xảy ra va chạm, Enemy nhận sát thương:

```
enemy.takeDamage(bullet.getDamage());
```

Đối với đạn thường, sau va chạm:

```
spawnBulletExplosion(...);  
bullet.deactivate();
```

Đối với đạn xuyên mục tiêu, hệ thống sử dụng:

```
bullet.hasAlreadyHit(enemy)  
bullet.markHit(enemy)
```

để đảm bảo một viên đạn xuyên chỉ gây sát thương một lần trên mỗi Enemy.

4.8.5 Bullet của Enemy va chạm Player

Nếu Bullet không thuộc Player, hệ thống kiểm tra:

```
bullet.intersects(player)
```

Khi va chạm xảy ra:

```
player.takeDamage(bullet.getDamage());
```

Như vậy, `Bullet.intersects()` đảm nhiệm kiểm tra hình học, trong khi `Player.takeDamage()` đảm nhiệm logic gameplay sau khi va chạm được xác nhận.

4.9 Va chạm Player với Item

4.9.1 Class và phương thức

- Class điều phối: `GameWorld`
- Phương thức: `updateItemCollection()`
- Class đối tượng: `Item`
- Phương thức kiểm tra: `Item.intersects()`

Mỗi Item chưa được thu thập được kiểm tra với Player:

```
if (!item.intersects(
    player.getPosition(),
    player.getRadius())) {
    continue;
}
```

Khi va chạm xảy ra, loại Item được xác định và hiệu ứng tương ứng được thực hiện. Ví dụ:

- `GoldItem`: cộng Gold;
- `GemItem`: cộng Gem;
- `BuffItem`: thêm Buff;
- `EnergyCrystal`: hồi Mana.

Cuối cùng:

```
item.collect();
```

đánh dấu Item đã được thu thập và Item sẽ được loại khỏi danh sách quản lý.

Loại va chạm	Class chính	Phương thức
Player – Map	Player / GameWorld	Player.move(), GameWorld.canMoveTo()
Player – Obstacle	GameWorld / Obstacle	canMoveTo(), Obstacle.intersectsCircle()
Enemy – Map	Enemy / GameWorld	Enemy.move(), Enemy.smartMoveTo(), GameWorld.canMoveTo()
Enemy – Enemy	Enemy	separateFromOtherEnemies()
Player – Enemy	GameWorld / Enemy	resolvePlayerEnemyCollisions(), Enemy.meleeAI()
Bullet – Wall	GameWorld / MapManager	updateBullets(), isBulletCollidingWithWall()
Bullet – Obstacle	GameWorld / Obstacle	updateBullets(), intersectsCircle()
Bullet – Enemy	GameWorld / Bullet	updateBullets(), Bullet.intersects()
Bullet – Player	GameWorld / Bullet	updateBullets(), Bullet.intersects()
Player – Item	GameWorld / Item	updateItemCollection(), Item.intersects()

Bảng 4.1: Các thành phần chính của hệ thống xử lý va chạm

4.10 Tổng hợp các phương thức xử lý va chạm

4.11 Đánh giá thiết kế

Thiết kế hiện tại phân chia trách nhiệm xử lý va chạm thành hai mức chính.

Mức thực thể chịu trách nhiệm cho các hành vi cục bộ, ví dụ:

- `Player.move()`;
- `Enemy.move()`;
- `Enemy.separateFromOtherEnemies()`;
- `Bullet.intersects()`;
- `Item.intersects()`.

Trong khi đó, **GameWorld** chịu trách nhiệm điều phối các tương tác giữa nhiều hệ thống thông qua:

- `canMoveTo()`;
- `resolvePlayerEnemyCollisions()`;
- `updateBullets()`;
- `updateItemCollection()`.

Cách tổ chức này giúp tránh việc một Entity phải trực tiếp quản lý toàn bộ các Entity khác trong game. Đồng thời, **GameWorld** đóng vai trò là trung tâm kết nối giữa hệ thống vật lý đơn giản, combat, map, item và hiệu ứng hình ảnh.

4.12 Kết luận

Hệ thống va chạm của *Spirit Knight* sử dụng chủ yếu mô hình va chạm hình tròn cho các Entity động và kết hợp với vùng va chạm của bản đồ cũng như Obstacle.

Các phép kiểm tra được thực hiện liên tục trong game loop, sau đó kết quả va chạm được chuyển thành các phản ứng gameplay như: ngăn di chuyển, đẩy thực thể ra khỏi nhau, gây sát thương, phá vật cản, vô hiệu hóa đạn hoặc thu thập vật phẩm.

Việc tập trung điều phối tại **GameWorld** đồng thời phân chia các phép kiểm tra cụ thể cho từng class giúp hệ thống tương đối dễ mở rộng khi bổ sung Enemy, Weapon, Bullet hoặc Item mới.

Ngoài ra còn một số xử lý va chạm khác như giữa tường, cổng(door) và player được xử lý trực tiếp trong MAP

Chương 5

Kĩ thuật Story cinematic

Hệ thống Story Cinematic trong *Spirit Knight* được xây dựng trên JavaFX theo mô hình chuỗi cảnh (*scene sequence*). Mỗi cinematic được chia thành nhiều cảnh nhỏ, trong đó mỗi cảnh chịu trách nhiệm cho một hiệu ứng hoặc một giai đoạn cụ thể của cốt truyện.

5.0.1 Điều phối cinematic theo chuỗi cảnh

Class `CinematicPlayer` được sử dụng để quản lý thứ tự thực thi của các cảnh. Các Controller chỉ cần đăng ký các phương thức cảnh theo đúng trình tự.

Ví dụ phần Intro:

```
cinematicPlayer
    .addScene(this::playBootScene)
    .addScene(this::playCodingScene)
    .addScene(this::playConsoleScene)
    .addScene(this::playSignalScene)
    .addScene(this::playCorruptionScene)
    .addScene(this::playBugScene)
    .addScene(this::playVisualNovelScene)
    .addScene(this::playPortalScene);
```

Mỗi scene nhận một callback `Runnable onFinished`. Sau khi animation của scene hiện tại hoàn tất, callback được gọi để chuyển sang scene tiếp theo.

Cách thiết kế này giúp cinematic không phụ thuộc vào một Timeline lớn duy nhất mà được chia thành nhiều thành phần nhỏ, dễ quản lý và mở rộng.

5.0.2 Sử dụng JavaFX Animation

Các hiệu ứng chuyển cảnh được xây dựng bằng hệ thống Animation của JavaFX, chủ yếu gồm:

- `FadeTransition`: tạo hiệu ứng xuất hiện và biến mất;
- `ScaleTransition`: tạo hiệu ứng phóng to hoặc thu nhỏ;
- `PauseTransition`: tạo khoảng nghỉ giữa các cảnh;

- **Timeline**: điều khiển các hiệu ứng lặp theo thời gian;
- **ParallelTransition**: chạy nhiều animation đồng thời.

Ví dụ, khi một khung hình Visual Novel xuất hiện, hình ảnh được fade từ trong suốt sang hiển thị, đồng thời lớp nền đen được fade ra ngoài.

```
FadeTransition imageFade = ...
FadeTransition blackFadeOut = ...
```

```
ParallelTransition transition =
    new ParallelTransition(
        imageFade,
        blackFadeOut
    );
```

```
transition.play();
```

Việc sử dụng **ParallelTransition** cho phép nhiều hiệu ứng xảy ra cùng lúc, tạo cảm giác chuyển cảnh tự nhiên hơn.

5.0.3 Hiệu ứng camera điện ảnh

Trong class **VisualNovelScene**, hình ảnh của mỗi story frame không hoàn toàn đứng yên mà sử dụng **ScaleTransition** để phóng to từ từ.

```
activeZoom =
    new ScaleTransition(
        Duration.seconds(12),
        storyImage
    );

activeZoom.setFromX(1.0);
activeZoom.setFromY(1.0);
activeZoom.setToX(1.05);
activeZoom.setToY(1.05);
```

Kỹ thuật này tạo hiệu ứng zoom chậm tương tự *Ken Burns Effect*, giúp các hình ảnh tĩnh có cảm giác cinematic và có chiều sâu hơn.

5.0.4 Hiệu ứng hội thoại Visual Novel

Class chính xử lý phần hội thoại là:

VisualNovelScene

Mỗi đoạn truyện được biểu diễn bởi một **StoryFrame**. Một frame có thể chứa:

- hình ảnh;
- tiêu đề;
- nội dung hội thoại;
- nhạc nền;
- thông tin thời gian chuyển nhạc.

Nội dung hội thoại không xuất hiện ngay lập tức mà sử dụng `TypingEffect` để hiển thị từng ký tự:

```
typingEffect.play(
    storyText,
    frame.text(),
    Duration.millis(28),
    callback
);
```

Kỹ thuật này mô phỏng cách hiển thị hội thoại thường thấy trong Visual Novel và các game nhập vai.

5.0.5 Kết hợp hiệu ứng hình ảnh và âm thanh

Cinematic sử dụng `SoundManager` để đồng bộ âm thanh với các sự kiện trên màn hình. Một số hiệu ứng được sử dụng gồm:

- âm thanh gõ bàn phím;
- âm thanh BIOS;
- tín hiệu Radio;
- âm thanh Bug;
- BGM riêng cho từng Story Frame.

Ví dụ:

```
SoundManager.getInstance()
    .playLoopSFX("Typing");
```

Âm thanh được dừng khi hiệu ứng tương ứng hoàn thành nhằm tránh nhiều âm thanh tiếp tục chạy chồng lên nhau.

5.0.6 Hiệu ứng Screen Shake và Flash

Để tăng cảm giác cinematic tại những sự kiện quan trọng, `StoryIntroController` sử dụng các hiệu ứng đặc biệt như:

```
ScreenShake.play(rootPane, 18, 12);
```

Kết hợp với các hiệu ứng flash đỏ, thay đổi opacity và scale, hệ thống tạo cảm giác lỗi chương trình hoặc thế giới game đang bị xâm nhập.

Ở cuối Intro, `PortalEffect` và hiệu ứng màn hình trắng được sử dụng làm cầu nối từ cinematic sang gameplay.

5.0.7 Tách dữ liệu cốt truyện khỏi mã nguồn

Nội dung story không được viết trực tiếp toàn bộ trong Controller. Thay vào đó, dữ liệu được đọc từ file JSON thông qua:

```
StoryConfigLoader.loadFromJson(  
    "/assets/story/story_intro.json"  
);
```

và:

```
StoryConfigLoader.loadFromJson(  
    "/assets/story/story_ending.json"  
);
```

Sau đó dữ liệu được chuyển thành các đối tượng `StoryFrame`.

Cách tổ chức này giúp phần nội dung và phần xử lý cinematic được tách biệt. Khi cần thay đổi lời thoại, hình ảnh hoặc BGM, có thể chỉnh sửa file cấu hình mà không cần thay đổi trực tiếp logic Java.

5.0.8 Cơ chế Skip và Continue

Cinematic cung cấp hai cơ chế điều khiển quan trọng:

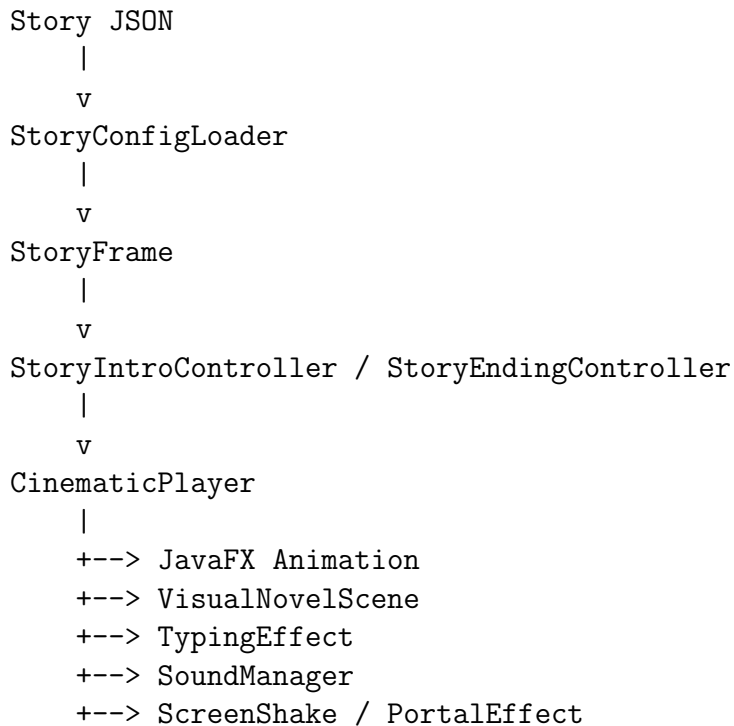
- `continueToNextFrame()`: chuyển sang frame tiếp theo của Visual Novel;
- `skipIntro()` hoặc `skipEnding()`: dừng toàn bộ cinematic.

Khi người chơi Skip, các animation, hiệu ứng chữ và âm thanh đang chạy đều được dừng trước khi chuyển sang trạng thái tiếp theo. Điều này giúp tránh tình trạng animation cũ vẫn tiếp tục hoạt động sau khi cinematic đã kết thúc.

5.0.9 Tổng kết

Story Cinematic của *Spirit Knight* được xây dựng bằng cách kết hợp nhiều kỹ thuật của JavaFX gồm animation theo thời gian, callback, Visual Novel, hiệu ứng chữ, âm thanh và cấu hình dữ liệu từ JSON.

Kiến trúc có thể được mô tả tổng quát như sau:



Việc tách nội dung, điều phối cinematic và hiệu ứng thành các thành phần riêng biệt giúp hệ thống dễ bảo trì và thuận tiện khi bổ sung các đoạn cốt truyện mới.

Chương 6

Hệ thống cơ sở dữ liệu

Hệ thống cơ sở dữ liệu trong *Spirit Knight* được sử dụng để lưu trữ và khôi phục trạng thái của người chơi giữa các lần chơi.

Các dữ liệu chính được quản lý bao gồm:

- thông tin tài khoản người chơi;
- tiến trình chơi như level, HP, energy, phòng hiện tại và điểm số;
- lượng Gold và Gem tích lũy;
- danh sách Hero, Pet, Weapon và Buff đã sở hữu;
- trang bị hiện tại và hai slot vũ khí của người chơi.

Việc truy cập cơ sở dữ liệu được tổ chức theo mô hình DAO (*Data Access Object*). Ví dụ, class `ShopDAO` chịu trách nhiệm thực hiện các truy vấn liên quan đến Shop, inventory, Gold, Gem, Buff và Weapon Loadout.

Các truy vấn được thực hiện bằng JDBC thông qua các thành phần:

`Connection`
`PreparedStatement`
`ResultSet`

Trong các thao tác quan trọng như mua vật phẩm, chương trình sử dụng transaction với `commit()` và `rollback()` nhằm đảm bảo dữ liệu không bị sai lệch khi một bước trong quá trình xử lý thất bại.

Class `EquipmentLoader` chịu trách nhiệm đọc dữ liệu trang bị từ database sau khi người chơi đăng nhập và đồng bộ lại các hệ thống trong game như:

- `WeaponSelectionManager`;
- `HeroSelectionManager`;
- `PetSelectionManager`;
- `BuffInventoryManager`.

Ngoài ra, class `UserSession` lưu thông tin tài khoản hiện đang đăng nhập, giúp các thành phần khác của game xác định `userId` và `username` hiện tại.

Nhìn chung, hệ thống database được tách khỏi logic gameplay thông qua các class chuyên trách, giúp dữ liệu người chơi được lưu trữ ổn định, dễ quản lý và thuận tiện khi mở rộng các chức năng Shop, Save Game và hệ thống trang bị.

6.1 Cơ chế Cache dữ liệu

Hệ thống *Spirit Knight* sử dụng bộ nhớ Cache trong RAM nhằm giảm số lần truy vấn Database và tăng tốc độ phản hồi của giao diện.

Class chính quản lý cache của người chơi là:

`PlayerSessionCache`

Cache lưu các dữ liệu thường xuyên được sử dụng như:

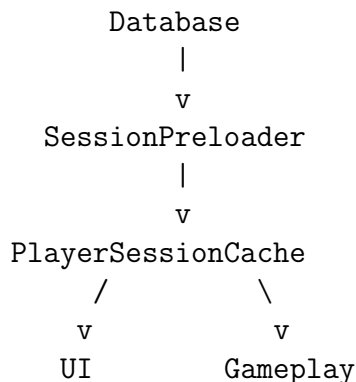
- Gold và Gem;
- Pet đã sở hữu;
- Weapon đã sở hữu;
- Hero đã sở hữu;
- số lượng Buff;
- trạng thái dữ liệu Shop và Equipment đã được tải.

Khi người chơi đăng nhập, class:

`SessionPreloader`

sử dụng `CompletableFuture` để tải song song nhiều nhóm dữ liệu từ Database. Sau khi hoàn thành, dữ liệu được đưa vào `PlayerSessionCache` để UI và gameplay có thể truy cập nhanh trực tiếp từ RAM.

Luồng xử lý có thể mô tả như sau:



Đối với bảng xếp hạng, game sử dụng một cache riêng:

LeaderboardCache

Khác với cache của người chơi tồn tại trong suốt session, Leaderboard chỉ được cache trong một khoảng thời gian ngắn. Nếu dữ liệu vẫn còn mới, chương trình sử dụng dữ liệu trong RAM thay vì truy vấn lại Database.

Nhờ cơ chế cache, các màn hình như Shop, HUD và Leaderboard có thể hiển thị dữ liệu nhanh hơn, đồng thời giảm lượng truy vấn không cần thiết tới Database và hạn chế hiện tượng giật hoặc chờ khi mở UI.

Chương 7

Hệ thống giao diện người dùng

Hệ thống giao diện của *Spirit Knight* được xây dựng bằng JavaFX, kết hợp giữa FXML, CSS và các Controller Java nhằm tách biệt phần thiết kế giao diện với logic xử lý.

Class trung tâm của hệ thống là:

`UIManager`

Class này chịu trách nhiệm quản lý và chuyển đổi giữa các màn hình chính của game như:

- Intro và Story Cinematic;
- Login và Register;
- Main Menu;
- HUD trong gameplay;
- Shop;
- Pause và Settings;
- Leaderboard và Account;
- Level Clear, Victory và Game Over.

Các giao diện được thiết kế bằng FXML và tải thông qua `FXMLLoader`. Sau khi tải, `UIManager` lưu lại Controller tương ứng để giao tiếp với từng màn hình.

Việc chuyển đổi giao diện được đồng bộ với trạng thái game thông qua:

`UIManager.handleStateChange(GameState state)`

Ví dụ, khi game ở trạng thái `PLAYING`, HUD được hiển thị; khi `PAUSED`, màn hình Pause được đặt phía trên gameplay; khi `GAME_OVER` hoặc `GAME_VICTORY`, giao diện kết quả tương ứng được kích hoạt.

7.0.1 HUD trong gameplay

HUD được cập nhật trực tiếp từ dữ liệu của `GameWorld`. Thông qua phương thức:

```
UIManager.updateHUD(GameWorld world)
```

giao diện có thể hiển thị các thông tin cần thiết của người chơi và trạng thái màn chơi. HUD đồng thời cung cấp các thao tác tương tác như:

- Pause game;
- chuyển đổi vũ khí;
- sử dụng Buff;
- theo dõi trạng thái gameplay.

7.0.2 Giao diện Shop

`ShopController` quản lý giao diện cửa hàng với các nhóm:

PET | WEAPON | HERO | BUFF

Các vật phẩm được tạo thành các card giao diện và thay đổi trạng thái hiển thị tùy theo việc vật phẩm đã được sở hữu, đang được trang bị hay chưa được mở khóa.

Shop cũng kết hợp với Database và Session Cache. Các tác vụ truy vấn database được thực hiện bất đồng bộ bằng `CompletableFuture` để hạn chế việc truy vấn dữ liệu làm đứng JavaFX UI Thread.

7.0.3 Animation và Loading UI

Ngoài các thành phần FXML thông thường, hệ thống còn sử dụng `Canvas` và `AnimationTimer` để tạo các thành phần giao diện động.

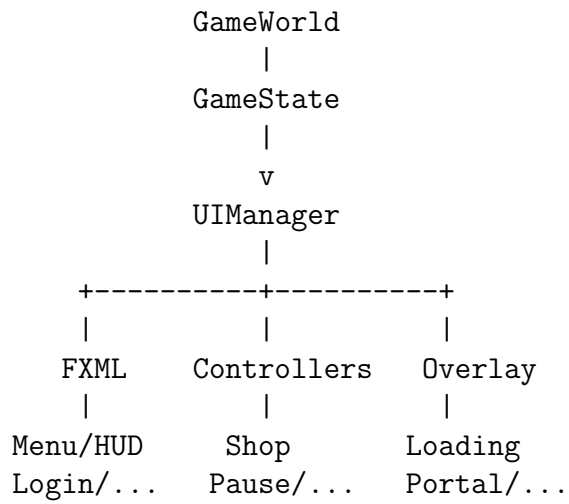
Ví dụ:

```
CatLoadingOverlay
```

hiển thị animation sprite của Pet khi hệ thống đang tải dữ liệu. Các frame được cắt từ Sprite Sheet và render liên tục trên `Canvas`.

7.0.4 Tổng kết

Kiến trúc UI có thể mô tả khái quát như sau:



Việc sử dụng một **UIManager** trung tâm kết hợp FXML, Controller, CSS, Canvas và Animation giúp hệ thống giao diện được tổ chức rõ ràng, thuận tiện cho việc chuyển đổi màn hình và mở rộng thêm các chức năng UI trong tương lai.

Chương 8

Hệ thống Buff

Buff là các hiệu ứng hỗ trợ tạm thời giúp người chơi tăng khả năng sống sót hoặc chiến đấu. Hệ thống Buff được xây dựng dựa trên class trừu tượng `Buff`, trong khi `BuffType` định nghĩa các loại Buff và thông số tương ứng.

Các Buff chính trong game gồm:

- **Energy Shield:** giảm 50% sát thương nhận vào trong 15 giây.
- **Health Core:** hồi ngay 40 HP cho người chơi.
- **Speed Module:** tăng 30% tốc độ di chuyển trong 12 giây.
- **Power Core:** tăng 25% sát thương gây ra trong 12 giây.
- **Chain Lightning:** tạo sét lan từ mục tiêu sang tối đa 2 Enemy ở gần.
- **Death Explosion:** Enemy có xác suất 50% phát nổ khi chết, gây sát thương và hiệu ứng cháy lên các Enemy xung quanh.
- **Dragon Breath:** định kỳ tạo đòn phun lửa, gây cháy và có khả năng kích nổ mục tiêu đang bị cháy.
- **Holy Nova:** bảo vệ Player khỏi sát thương chí tử, hồi 50% HP tối đa và tạo một đợt Holy Nova mạnh.

Về mặt kỹ thuật, class `Buff` quản lý vòng đời chung của mỗi hiệu ứng thông qua các phương thức:

```
activate()  
tick()  
remove()
```

Trong đó, `activate()` kích hoạt Buff, `tick()` cập nhật thời gian còn lại theo từng frame và `remove()` loại bỏ hiệu ứng khỏi Player.

Một số Buff chiến đấu còn có thể phản ứng với các sự kiện như:

```
onDamageDealt()  
onEnemyKilled()
```

Nhờ sử dụng class cơ sở **Buff** kết hợp với **BuffType**, mỗi **Buff** có thể triển khai cơ chế riêng nhưng vẫn tuân theo cùng một vòng đời quản lý. Thiết kế này giúp hệ thống dễ mở rộng khi cần bổ sung thêm các hiệu ứng mới.

Chương 9

Hệ thống Weapon

Hệ thống Weapon trong *Spirit Knight* quản lý các loại vũ khí mà Player có thể sử dụng trong quá trình chiến đấu. Hệ thống được thiết kế theo hướng kế thừa và đa hình, với class cơ sở:

Weapon

Class này quản lý các thuộc tính chung của vũ khí như:

- sát thương (**damage**);
- thời gian hồi chiêu (**cooldown**);
- Mana tiêu hao;
- hình ảnh và âm thanh;
- phương thức tấn công.

Các vũ khí được chia thành hai nhóm cơ bản là vũ khí tầm xa **Gun** và vũ khí cận chiến **Melee**.

9.0.1 Vũ khí tầm xa

Class **Gun** tạo các đối tượng **Bullet** bay theo hướng từ Player đến vị trí ngắm. Mỗi viên đạn chứa các thông tin như:

```
position
velocity
damage
radius
owner
```

Ngoài đạn thông thường, hệ thống còn hỗ trợ các đặc tính đặc biệt như đạn xuyên mục tiêu, phản xạ, phát nổ khi chạm địa hình và đạn Sound Wave.

9.0.2 Vũ khí cận chiến

Class `Melee` xử lý các vũ khí cận chiến. Thay vì tạo `Bullet`, hệ thống xác định một vùng hình quạt theo hướng tấn công và gây sát thương lên `Enemy` nằm trong phạm vi đó.

Phương thức chính:

```
GameWorld.damageEnemiesInArc(...)
```

được sử dụng để xác định các `Enemy` trúng đòn, đồng thời tạo hiệu ứng chém tương ứng trên màn hình.

9.0.3 Vũ khí tích năng

Một số `Weapon` đặc biệt sử dụng cơ chế giữ nút để tích năng và thả nút để tấn công, tiêu biểu gồm:

```
PrototypeRailgun  
IonElectromagneticGun
```

Mức tích năng có thể ảnh hưởng đến sát thương, kích thước đạn, Mana tiêu hao, khả năng chí mạng hoặc các thuộc tính đặc biệt của projectile.

`PrototypeRailgun` có khả năng tạo nhiều projectile, xuyên mục tiêu và phản xạ khi mức charge tăng.

`IonElectromagneticGun` tạo `Ion Orb` có sát thương, kích thước và bán kính vụ nổ tăng theo mức charge.

9.0.4 Quản lý loại vũ khí

Các loại `Weapon` và thông số được định nghĩa tập trung trong:

```
WeaponType
```

Mỗi loại chứa các thông tin như tên, sprite, âm thanh, damage, cooldown, tốc độ `Bullet`, tầm đánh, giá và Mana tiêu hao.

Phương thức:

```
WeaponType.createWeapon()
```

được sử dụng để tạo đối tượng `Weapon` tương ứng, giúp tập trung quá trình khởi tạo và thuận tiện khi bổ sung vũ khí mới.

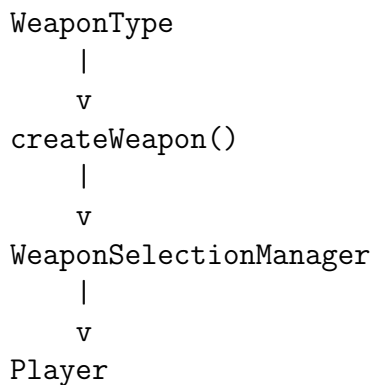
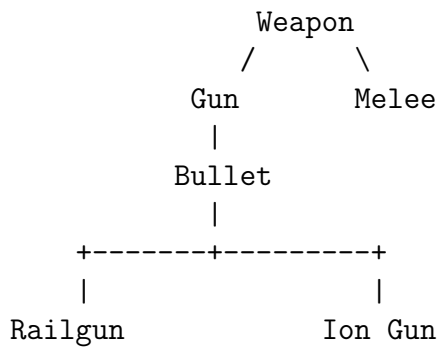
9.0.5 Quản lý trang bị

Player sử dụng hai slot Weapon được quản lý bởi:

WeaponSelectionManager

Hai slot xác định các Weapon được trang bị trước khi bắt đầu game. Người chơi có thể chuyển đổi giữa hai vũ khí trong quá trình gameplay.

Có thể mô tả kiến trúc tổng quát như sau:



Nhìn chung, hệ thống Weapon được chia thành các thành phần độc lập cho thông số, hành vi tấn công, projectile và quản lý trang bị. Thiết kế này giúp dễ dàng bổ sung thêm vũ khí và cơ chế chiến đấu mới mà không cần thay đổi toàn bộ hệ thống.

Chương 10

Hệ thống Pet

Hệ thống Pet trong *Spirit Knight* quản lý các nhân vật đồng hành hỗ trợ Player trong quá trình khám phá Dungeon và chiến đấu.

Class chính của hệ thống là:

Pet

Mỗi Pet được quản lý bởi một State Machine gồm các trạng thái:

IDLE
FOLLOWING
CATCHING_UP
CHASING
ATTACKING
STUCK

10.0.1 Cơ chế di chuyển và theo Player

Pet sử dụng cơ chế *breadcrumb* để ghi lại các vị trí mà Player đã đi qua. Thay vì luôn di chuyển theo đường thẳng trực tiếp đến Player, Pet có thể lần lượt đi qua các waypoint đã được ghi lại.

Phương thức chính:

```
Pet.update(...)  
Pet.recordPlayerPath(...)  
Pet.moveSmart(...)
```

Khi Pet ở quá xa Player, tốc độ di chuyển được tăng lên để bắt kịp. Nếu khoảng cách quá lớn hoặc Pet bị kẹt trong thời gian dài, hệ thống tự động dịch chuyển Pet về một vị trí an toàn gần Player.

10.0.2 AI chiến đấu

Pet có khả năng tự động phát hiện và tấn công Enemy.

Các phương thức chính gồm:

```
findNearestEnemy()  
updateEnemyChase()  
updateCombat()  
attackEnemy()
```

Pet tìm Enemy gần nhất trong phạm vi cho phép. Khi phát hiện mục tiêu, Pet chuyển sang trạng thái **CHASING** và tự động di chuyển đến Enemy.

Khi Enemy nằm trong tầm đánh, Pet chuyển sang:

ATTACKING

và gây sát thương theo thời gian hồi chiêu.

Pet đồng thời bị giới hạn khoảng cách chiến đấu so với Player, tránh trường hợp Pet đuổi theo Enemy quá xa và tách khỏi nhân vật.

10.0.3 Quản lý loại Pet

Các loại Pet được định nghĩa tập trung trong:

PetType

Mỗi loại Pet chứa các thông tin như:

- tên hiển thị;
- Sprite Sheet Idle và Run;
- âm thanh;
- số frame animation;
- kích thước render;
- tốc độ di chuyển;
- thời gian mỗi frame;
- giá mua.

Đối tượng Pet được khởi tạo thông qua:

PetFactory

Factory nhận **PetType** và tạo Pet tại vị trí tương đối so với Player. Cách tổ chức này giúp tách logic khởi tạo Pet khỏi Shop và Gameplay.

10.0.4 Animation

Pet sử dụng Sprite Sheet để hiển thị animation. Pet cũng tự động lật Sprite theo hướng di chuyển hoặc hướng của Enemy đang tấn công.

10.0.5 Pet khi chuyển phòng

Khi Player bước vào một phòng chiến đấu, hệ thống sử dụng:

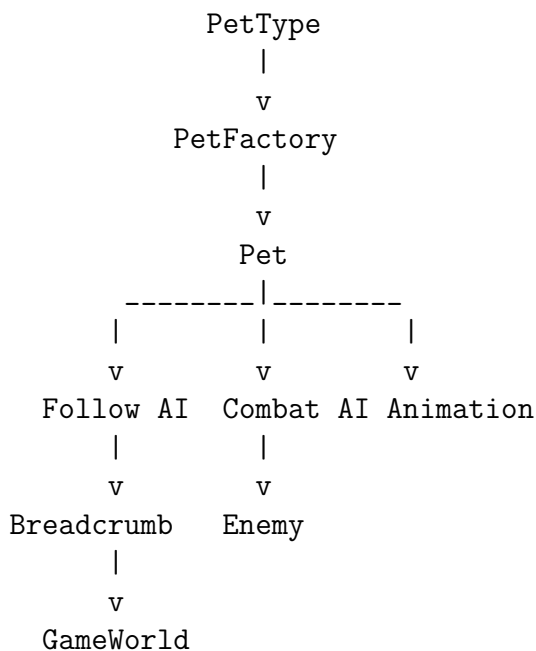
`PetRoomEntryController`

để chờ Pet đi vào phòng trước khi đóng cửa và bắt đầu Wave.

Nếu Pet bị kẹt hoặc không thể vào phòng trong khoảng thời gian cho phép, hệ thống tự động đưa Pet vào vị trí an toàn bên trong phòng.

10.0.6 Tổng kết

Kiến trúc Pet có thể mô tả khái quát như sau:



Nhìn chung, hệ thống Pet kết hợp State Machine, AI theo dấu Player, tìm đường tránh vật cản, AI chiến đấu và Sprite Sheet Animation. Nhờ đó Pet hoạt động như một thực thể đồng hành độc lập thay vì chỉ đơn thuần di chuyển theo vị trí của Player.

Chương 11

Hệ thống Item

Hệ thống Item trong *Spirit Knight* quản lý các vật phẩm xuất hiện và được thu thập trong quá trình gameplay.

Class cơ sở của hệ thống là:

`Item`

Class này quản lý các thuộc tính chung như:

- tên vật phẩm;
- vị trí trong `GameWorld`;
- bán kính va chạm;
- hình ảnh hiển thị;
- trạng thái đã được thu thập;
- chuyển động và hiệu ứng hút vật phẩm.

Các loại Item chính trong game gồm:

- `GoldItem`: cung cấp Gold cho người chơi;
- `GemItem`: cung cấp Gem;
- `EnergyCrystal`: phục hồi Energy/Mana;
- `BuffItem`: cung cấp các Buff hỗ trợ chiến đấu.

11.0.1 Thu thập Item

Va chạm giữa Player và Item được kiểm tra thông qua:

```
Item.intersects(...)
```

Khi Player chạm vào Item, phương thức:

```
Item.collect()
```

đánh dấu vật phẩm đã được thu thập và có thể phát sinh `GameEvent` tương ứng.

11.0.2 Cơ chế hút Item

Một đặc điểm của hệ thống là cơ chế tự động hút vật phẩm về phía Player. Khi Player tiến vào phạm vi cho phép, Item sẽ tự động di chuyển về phía nhân vật.

Hệ thống sử dụng:

`ItemMagnetSystem`

để quản lý bán kính và tốc độ hút riêng cho từng loại Item.

Tốc độ hút tăng dần khi Item tiến gần Player, giúp quá trình thu thập vật phẩm diễn ra nhanh và tự nhiên hơn.

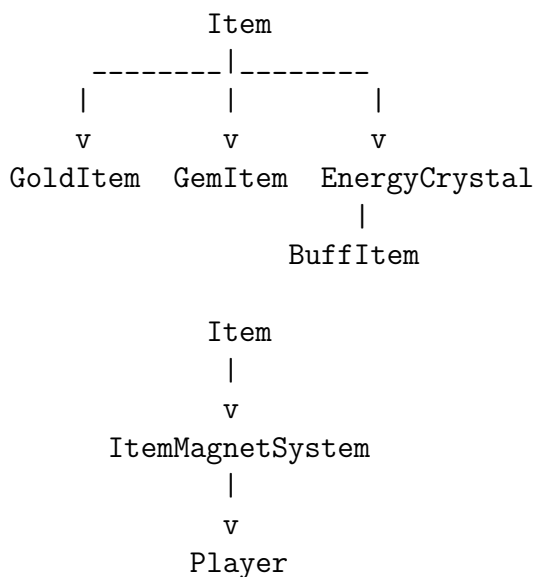
11.0.3 Hiệu ứng hiển thị

Item được render trực tiếp lên GameWorld thông qua JavaFX Canvas. Các vật phẩm sử dụng sprite tương ứng và có hiệu ứng thay đổi kích thước nhẹ theo thời gian.

`BuffItem` còn sử dụng hiệu ứng dao động lên xuống và phát sáng để giúp người chơi dễ nhận biết các vật phẩm Buff quan trọng trên bản đồ.

11.0.4 Tổng kết

Kiến trúc Item có thể mô tả khái quát như sau:



Nhìn chung, hệ thống Item kết hợp cơ chế kế thừa, kiểm tra va chạm, Sprite Animation và Magnet System. Thiết kế này giúp việc bổ sung các loại vật phẩm mới tương đối đơn giản và đồng thời cải thiện trải nghiệm thu thập vật phẩm trong quá trình chiến đấu.